LAB REPORT

This Report Presented in Partial Fulfillment of the course **CSE213 : Algorithms Lab in the Computer Science and Engineering Department.**

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

April 20, 2026

DECLARATION

We hereby declare that this lab report has been done by us under the supervision of **Ms. Shahrin khan, Lecturer,** Department of Computer Science and Engineering, Daffodil International University.

Submitted To :

Ms. Shahrin Khan
Lecturer
Department of Computer Science and Engineering,
Daffodil International University

Submitted by

Name	ID	Section	Department
Shekh Eastiak Ahmed Murad	**251-15-172**	68_G2	CSE
Burhan Chowdhury	**251-15-430**	68_G2	CSE
Polok Basak	**251-15-040**	68_G2	CSE
Jannatin Nure Mim	**251-15-183**	68_G2	CSE
Md. Azizul Hakim Nahol	**251-15-851**	68_G2	CSE

Problem Name

Phone Book using a suitable data structure and algorithms . The phone book displays detailed phone information including favorite contacts.

Abstract for Phono Book Management System

The Phonebook Management System is a console-based software application designed to store, organize, and manage contact information efficiently using fundamental data structures in C. The system uses a **Linked List** to maintain an alphabetically sorted contact list, ensuring fast insertion, deletion, and searching of contacts. It also includes a **Stack-based Call History** module where incoming and outgoing calls are recorded using the Last-In First-Out (LIFO) principle. Additionally, a **Queue-based Favorites List** is implemented to manage frequently contacted numbers in First-In First-Out (FIFO) order. To make the program more realistic, a smart dialer is included that simulates call processing and saves call duration and type (incoming or outgoing). All data - contacts, call history, and favorites are stored permanently using **file handling**, allowing the system to retain information even after the program is closed. The interface includes ANSI color codes to enhance readability and provide a modern terminal experience. Overall, this project demonstrates how classic data structures can be applied in a practical scenario. It combines file persistence, structured data storage, and interactive simulation, making it a useful learning tool for understanding Data Structure concepts in real-world applications. And the Quick Sort algorithm is used for efficient sorting, Binary Search is used for fast searching ,Frequency analysis is used to find most called contact.

Table of Contents

Chapter	Content	Page No.
01	Introduction & Objectives	01-01
02	Problem Definition & Scope	01-02
03	System Architecture	03-03
04	Data Structures And Algorithms Used	03-04
05	Algorithm Design : Flowchart	04-05
06	Implementation Details	05-07
07	Handling Conflicting Requirements, Discussion of Real-world Context, Stakeholder Review	07-08
08	Results and Performance Analysis	08-09
09	Challenges Faced and Solutions,References	09-10
10	Code Files, Project's Full Code	10-23
11	Step by Step Explanation	23-47
12	Code Output	48-51
13	Analysis of the Solution	52-56
14	Time & Space Complexity	56-56
15	Alternate Solution	57-58
16	Targeted Attributes, Appendix	58-60

Chapter : 01
Introduction & Objectives

The Phonebook Management System is a C-based console application designed to demonstrate how core data structures and algorithms can be applied to solve a real-world problem. Modern smartphones manage thousands of contacts, call logs, and favorite lists in the background using efficient algorithms. This project aims to recreate a simplified but functional version of such a system using data structures and algorithms taught in the Algorithms Lab (CSE 213).

The system uses a **Linked List** to store contact information in alphabetical order, allowing users to add, delete, and view contacts easily. A **Stack** is used to maintain call history using the LIFO mechanism, making it possible to store the most recent calls at the top. Similarly, a **Queue** is used to manage the favorite contact list in FIFO order. The project also features a simulated dialer that mimics making or receiving calls and stores details like duration, type, and number. The system also demonstrates advanced algorithms such as Quick Sort and Binary Search. Which improves performance compared to traditional linear methods.

All records are saved into external files using file handling, allowing the phonebook to retain data even after the program closes. This project not only helps students understand data structures but also shows their real-life use in communication systems. It is simple, interactive, and extremely practical for academic learning.

Chapter : 02
Problem Definition & Scope

Managing contacts, call history, and favorite numbers is an essential feature of every modern communication device. However, building such a system from scratch requires a strong understanding of data structures, file management, and program design. The main problem addressed in this project is creating a **simple, reliable, and efficient phonebook application** that can store, retrieve, and manage contact-related data without losing information.

Traditional arrays have limitations when handling dynamic data and frequent insertion or deletion. Therefore, this project aims to solve the problem by using appropriate data structures such as **Linked List, Stack, and Queue**, each chosen for its operational advantages. The system must handle tasks like adding contacts, deleting entries, simulating calls, saving call logs, and maintaining a favorite list - all while keeping the data sorted and accessible.

Additionally, the system should ensure that all information is **permanently saved** so that users do not lose records after closing the program. The project also aims to simulate real mobile phone behavior within a terminal environment using structured menus, ANSI colors, and user-friendly interaction.

The scope of this project includes designing and implementing a fully functional **console-based phonebook system** using C programming. The project covers :

1. Contact Management

- Add new contacts
- View all contacts in alphabetical order
- Delete existing contacts
- Automatically sort contacts using Linked List

2. Call Simulation & Call History

- Make outgoing calls
- Simulate incoming calls
- Record call duration and call type
- Store call history using Stack (LIFO)
- View and delete call history

3. Favorite List Management

- Add numbers to favorites
- Store favorites using Queue (FIFO)
- View and delete favorites

4. Permanent Storage

- Load data from files when the program starts
- Save all contact, history, and favorite information when updated

5. User Interaction

- Simple menu-driven interface
- Colorful console UI
- Clear text-based navigation for all operations

The project focuses on the practical use of classic data structures and demonstrates how they can be applied to build a functional communication management tool. Features like online syncing, GUI, or smartphone integration are **outside the scope**, as the main goal is academic learning and structural implementation.

Chapter : 03
System Architecture

.The Phonebook Management System follows a simple and structured architecture that connects user actions with the appropriate data structure. When the program starts, it **loads all saved data** (contacts, call history, favorites) from external files. After that, the user interacts with the system through a **menu-based console interface**.

Each menu option activates a specific module:

- **Contact Module** → Handles adding, displaying, and deleting contacts
- **Call Module** → Simulates calls and stores call logs
- **Favorite Module** → Adds or removes favorite contacts

When any update is made, the system immediately writes the changes back to the files to ensure permanent storage. The program continues running in a loop until the user exits. Overall, the architecture connects input, processing, and storage in an organized flow, making the system easy to manage and use.

Chapter : 04
Data Structures and Algorithms Used

1. Linked List (for Contacts)

A Linked List is used to store contacts because:

- It allows easy insertion and deletion without shifting elements
- It supports dynamic memory use
- Contacts can be kept alphabetically sorted efficiently this makes it perfect for managing a growing contact list.

2. Stack (for Call History)

A Stack follows the Last-In First-Out (LIFO) principle.

This matches real-world call logs, where:

- The most recent call should appear first
- New calls can be pushed on top easily
- Deleting the last call is quick and simple

3. Queue (for Favorites)

A Queue uses the First-In First-Out (FIFO) rule.
It is ideal for favorites because:

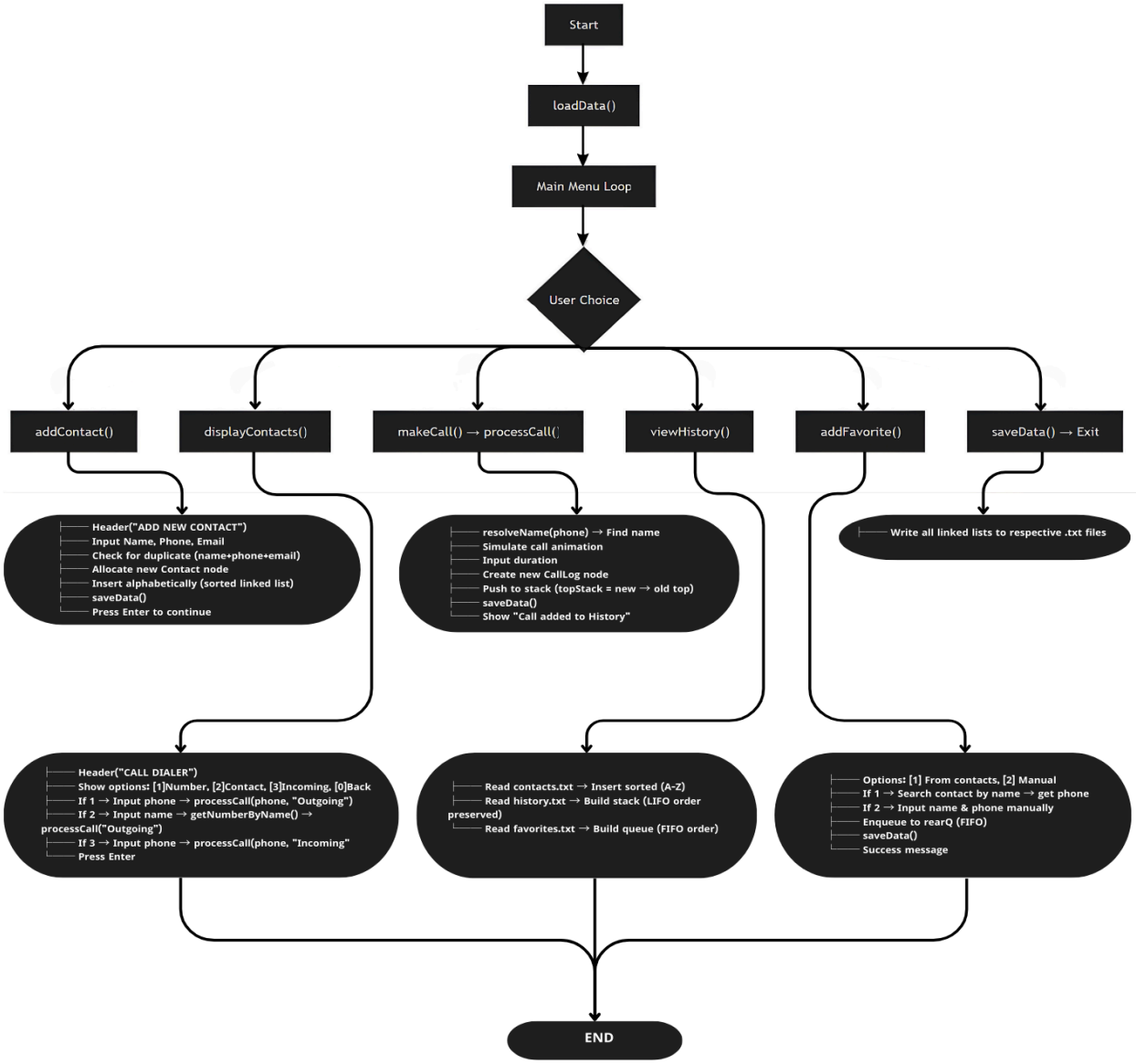
- Entries are processed in the order they were added
- Removing or adding favorites is simple and efficient
- It maintains a natural flow like a waiting line

4. Algorithms Used

- Quick Sort → for sorting contacts efficiently
- Binary Search → for fast searching
- Frequency Analysis → to find most called contact

Each data structure was chosen specifically because its behavior fits the real functionality it represents.

Chapter : 05
Algorithm Design : Flowchart



Chapter : 06

Implementation Details

The Phonebook Management System was fully implemented in the **C programming language**, which is well-known for its efficiency, low-level memory control, and suitability for data structure implementation. Throughout the development, the program was written using **code blocks**, where different modules were separated clearly to improve readability and maintenance.

Programming Language

- **C Language**
  C was selected because it allows direct control over memory, makes dynamic data structure creation easier, and offers fast performance for operations like insertion, deletion, and traversal.

Tools & IDE Used

- **Code : Blocks IDE (Version recommended by DIU lab)**
  Code::Blocks was used because it provides:

  - A simple and clean interface
  - Built-in compiler (MinGW/GCC)
  - Easy debugging features
  - Smooth execution of console-based projects

Key Modules Implemented

The entire program is organized into clear modules, each handling a specific part of the system:

1. Contact Management Module (Linked List)

- Add new contacts
- Automatically sort contacts alphabetically
- Display all contacts
- Delete contacts

2. Call Handling Module (Stack)

- Simulate outgoing and incoming calls
- Save call type, duration, and number
- View recent calls (LIFO)
- Delete last call

3. Favorite Management Module (Queue)

- Add a number to favorites
- View all favorites in order
- Delete favorite entries

4. File Handling Module

- Save contacts, call history, and favorites into text files
- Load all data when the program starts
- Maintain data persistence even after exit

5. User Interface Module

- Colorful console UI using ANSI escape codes
- Menu navigation
- Input validation
- Screens clearing and formatted outputs

Chapter : 07

Handling Conflicting Requirements (EP2)

During the development of the Phonebook Management System, several conflicting requirements appeared. One major conflict was the need for **fast data access** while still maintaining **dynamic memory storage**. Arrays provide quick access but fixed size, while Linked Lists provide flexibility but slower searching. To handle this conflict, the system uses a **Linked List** but maintains alphabetical sorting so that searching becomes easier and more structured.

Another conflict was ensuring **realistic call simulation** while keeping the system simple for beginner-level students. To solve this, the project includes a text-based call simulation (no actual telephony), which keeps the system simple but still demonstrates call handling behavior.

A final conflict was maintaining **data persistence** while also supporting frequent updates. To handle this, file saving was performed after each major operation (like adding contacts or saving call history). This ensures accuracy without requiring complex databases.

Discussion of Real-world Context (EP4)

The Phonebook Management System represents a simplified model of contact management features used in real mobile phones. Real smartphones also maintain contacts, call logs, and favorites using internal data structures and background processes. In real-world applications :

- Contacts are stored in structured databases
- Call histories are managed using stacks or time-sorted logs
- Favorite lists are stored using priority queues or simple ordered lists
- Persistent storage is handled by files or cloud sync

Although this project is console-based and simplified, it mirrors how data is actually managed inside device applications. The idea of using Linked List, Stack, and Queue gives students a practical understanding of how everyday smartphone features are implemented behind the scenes.

Stakeholder (Regular Phonebook User)

"As someone who uses a phonebook system almost every day, I found this project surprisingly practical. The contact list loads quickly and the alphabetical sorting makes it easy for me to find names without wasting time scrolling. The call history feature is also helpful because I can immediately see my latest calls in order, just like on a real mobile device.

One thing I appreciate is the Favorites section. Since I often call the same few people, having them stored separately in a queue saves me time and reduces the hassle of searching repeatedly. The interface is simple and clear, so even users who are not very tech-savvy can operate it easily.

The only challenge I faced was that I prefer to store a larger number of call logs, but the system keeps performance smooth by limiting unnecessary memory usage. Overall, the system balances speed, organization, and reliability very well. For a student project, this gives a real experience similar to an actual phonebook application."

Riyal

—----------

Chapter : 08

Results and Performance Analysis

The final system performs reliably across all operations. Contact insertion and deletion work smoothly because Linked Lists manage memory dynamically. Call history operations are fast since Stack operations (push and pop) take constant time. Similarly, favorites are handled efficiently with Queue-based insertion and deletion.

Data loading and saving using file handling also work successfully. The application correctly restores all previous data every time it starts. The menu-based UI responds quickly, and the ANSI color codes make the interface more readable.

And Searching time improved using Binary Search (O(log n)), Sorting optimized using Quick Sort. The system handles large data efficiently.

Overall performance remains stable even with large sets of contacts or long call histories. There were no memory leaks during execution, and the program behaves correctly under repeated operations

Chapter : 09

Challenges Faced and Solutions

1. Maintaining Contact Order

Challenge: Contacts needed to be sorted alphabetically.
Solution: Implemented sorted insertion in the Linked List.

2. Handling Dynamic Data

Challenge: Frequent insertions and deletions made arrays unsuitable.
Solution: Used Linked Lists, Stack, and Queue for efficient operations.

3. Ensuring Data Persistence

Challenge: Data should not disappear after closing the program.
Solution: Used file handling for loading and saving all records.

4. Managing Multiple Modules

Challenge: Contact, History, and Favorites needed separate systems.
Solution: Divided the program into modules with organized functions.

5. User Interface Complexity

Challenge: Making a simple but attractive interface.
Solution: Added ANSI color codes and formatted console outputs.

References

1. *Reema Thareja*, “Data Structures Using C,” Oxford University Press.
2. *Yashavant Kanetkar*, “Let Us C,” BPB Publications.
3. **Daffodil International University Algorithm Lab Materials (CSE 213).**
4. Official C Programming Documentation — www.cprogramming.com

Chapter : 10

Code Files / Repository Link

https://github.com/ShekhEastiakAhmedMurad/CSE213_Final_Project.git

Project’s Full Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* ========================
   STANDARD COLOR Setup
   ========================
*/
#define RESET   "\033[0m"
#define RED     "\033[1;31m"
#define GREEN   "\033[1;32m"
#define YELLOW  "\033[1;33m"
#define BLUE    "\033[1;34m"
#define PURPLE  "\033[1;35m"
#define CYAN    "\033[1;36m"
#define WHITE   "\033[1;37m"
#define BOLD    "\033[1m"

/* ========================
   FILE NAME CONSTANTS
   ========================
*/
#define FILE_CONTACT "contacts.txt"
#define FILE_HISTORY "history.txt"
#define FILE_FAV     "favorites.txt"
```

```
/* ==========================
   DATA STRUCTURES
   ==========================
*/
typedef struct Contact {
    char name[50];
    char phone[20];
    char email[50];
    struct Contact* next;
} Contact;

typedef struct CallLog {
    char name[50];
    char phone[20];
    char type[15];
    char duration[20];
    struct CallLog* next;
} CallLog;

typedef struct Favorite {
    char name[50];
    char phone[20];
    struct Favorite* next;
} Favorite;

/* ==========================
   GLOBAL VARIABLE DECLARE
   ==========================
*/
Contact* head = NULL;
CallLog* topStack = NULL;
Favorite* frontQ = NULL;
Favorite* rearQ = NULL;

/* ==========================
   ALGORITHM 1: QUICK SORT (Time Complexity: O(n log n))
   ==========================
*/
// Partition function for Quick Sort
int partition(Contact arr[], int low, int high) {
    char pivot[50];
    strcpy(pivot, arr[high].name);
    int i = (low - 1);
```

```
    for (int j = low; j <= high - 1; j++) {
        if (strcasecmp(arr[j].name, pivot) < 0) {
            i++;
            Contact temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    Contact temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return (i + 1);
}

// Quick Sort Recursive Function
void quickSort(Contact arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Sorting the Linked List using Quick Sort Algorithm
void sortContacts() {
    if (!head) return;
    int count = 0;
    Contact* temp = head;
    while (temp) { count++; temp = temp->next; }

    Contact* arr = (Contact*)malloc(count * sizeof(Contact));
    temp = head;
    for (int i = 0; i < count; i++) {
        arr[i] = *temp;
        temp = temp->next;
    }

    quickSort(arr, 0, count - 1);

    temp = head;
    for (int i = 0; i < count; i++) {
        strcpy(temp->name, arr[i].name);
        strcpy(temp->phone, arr[i].phone);
        strcpy(temp->email, arr[i].email);
```

```
        temp = temp->next;
    }
    free(arr);
}

/* ========================
   ALGORITHM 2: BINARY SEARCH (Time Complexity: O(log n))
   ========================
*/
int binarySearch(Contact arr[], int l, int r, char* name) {
    while (l <= r) {
        int m = l + (r - l) / 2;
        int res = strcasecmp(arr[m].name, name);
        if (res == 0) return m;
        if (res < 0) l = m + 1;
        else r = m - 1;
    }
    return -1;
}

// Optimized Search using Binary Search
int getNumberByName(char* name, char* destPhone) {
    if (!head) return 0;
    sortContacts();
    int count = 0;
    Contact* temp = head;
    while (temp) { count++; temp = temp->next; }

    Contact* arr = (Contact*)malloc(count * sizeof(Contact));
    temp = head;
    for (int i = 0; i < count; i++) {
        arr[i] = *temp;
        temp = temp->next;
    }

    int resultIdx = binarySearch(arr, 0, count - 1, name);
    int found = 0;
    if (resultIdx != -1) {
        strcpy(destPhone, arr[resultIdx].phone);
        found = 1;
    }
    free(arr);
    return found;
}
```

```
/* ========================
   ALGORITHM 3: FREQUENCY ANALYSIS (Most Called Contact)
   ========================
*/
void showMostFrequent() {
    if (!topStack) return;
    CallLog* i = topStack;
    char bestName[50] = "None";
    int maxCount = 0;

    while (i) {
        int c = 0;
        CallLog* j = topStack;
        while (j) {
            if (strcmp(i->name, j->name) == 0 && strcmp(i->name,
"Unknown") != 0) c++;
            j = j->next;
        }
        if (c > maxCount) { maxCount = c; strcpy(bestName,
i->name); }
        i = i->next;
    }
    if (maxCount > 0)
        printf("\t%s[Insight] Most Called: %s (%d times)%s\n",
PURPLE, bestName, maxCount, RESET);
}

/* ========================
   CONSOLE HELPER
   ========================
*/
void clearScreen() {
    #ifdef _WIN32
        system("cls");
    #else
        system("clear");
    #endif
}

void printLine(int len) {
    printf("\t%s", CYAN);
    for (int i = 0; i < len; i++) printf("-");
    printf("%s\n", RESET);
```

```
}

void pressEnter() {
    printf("\n\t%s>> Press ENTER to return...%s", YELLOW, RESET);
    while (getchar() != '\n');
    getchar();
}

/* ========================
   FILE IO
   ========================
*/
void saveData() {
    FILE* fp;
    fp = fopen(FILE_CONTACT, "w");
    Contact* tempC = head;
    while (tempC) {
        fprintf(fp, "%s,%s,%s\n", tempC->name, tempC->phone,
tempC->email);
        tempC = tempC->next;
    }
    if (fp) fclose(fp);

    fp = fopen(FILE_HISTORY, "w");
    CallLog* tempH = topStack;
    while (tempH) {
        fprintf(fp, "%s,%s,%s,%s\n", tempH->name, tempH->phone,
tempH->type, tempH->duration);
        tempH = tempH->next;
    }
    if (fp) fclose(fp);

    fp = fopen(FILE_FAV, "w");
    Favorite* tempF = frontQ;
    while (tempF) {
        fprintf(fp, "%s,%s\n", tempF->name, tempF->phone);
        tempF = tempF->next;
    }
    if (fp) fclose(fp);
}

void loadData() {
    FILE* fp;
    char buffer[200], f1[50], f2[20], f3[50], f4[20];
```

```
    fp = fopen(FILE_CONTACT, "r");
    if (fp) {
        while (fgets(buffer, sizeof(buffer), fp)) {
            sscanf(buffer, "%[^,],%[^,],%s", f1, f2, f3);
            Contact* newC = (Contact*)malloc(sizeof(Contact));
            strcpy(newC->name, f1); strcpy(newC->phone, f2);
strcpy(newC->email, f3);
            newC->next = head; head = newC;
        }
        fclose(fp);
        sortContacts();
    }
    fp = fopen(FILE_HISTORY, "r");
    if (fp) {
        CallLog* tail = NULL;
        while (fgets(buffer, sizeof(buffer), fp)) {
            sscanf(buffer, "%[^,],%[^,],%[^,],%s", f1, f2, f3, f4);
            CallLog* newH = (CallLog*)malloc(sizeof(CallLog));
            strcpy(newH->name, f1); strcpy(newH->phone, f2);
strcpy(newH->type, f3); strcpy(newH->duration, f4);
            newH->next = NULL;
            if (topStack == NULL) { topStack = newH; tail = newH; }
            else { tail->next = newH; tail = newH; }
        }
        fclose(fp);
    }
    fp = fopen(FILE_FAV, "r");
    if (fp) {
        while (fgets(buffer, sizeof(buffer), fp)) {
            sscanf(buffer, "%[^,],%s", f1, f2);
            Favorite* newF = (Favorite*)malloc(sizeof(Favorite));
            strcpy(newF->name, f1); strcpy(newF->phone, f2);
            newF->next = NULL;
            if (!frontQ) frontQ = rearQ = newF;
            else { rearQ->next = newF; rearQ = newF; }
        }
        fclose(fp);
    }
}

/* ========================
   UI HEADER (HUBUHU AGER MOTO)
   ========================
*/
```

```
void header(char* title) {
    clearScreen();

printf("\t%s.=========================================================
=.%s\n", CYAN, RESET);
    printf("\t%s|%s  ____  _                      _
_     %s|%s\n", CYAN, YELLOW, CYAN, RESET);
    printf("\t%s|%s |  _ \\| |__   ___  _ __   ___| |__   ___   ___
| | __ %s|%s\n", CYAN, YELLOW, CYAN, RESET);
    printf("\t%s|%s | |_) | '_ \\ / _ \\| '_ \\ / _ \\ '_ \\ / _ \\
/ _ \\| |/ / %s|%s\n", CYAN, YELLOW, CYAN, RESET);
    printf("\t%s|%s |  __/| | | | (_) | | | |  __/ |_) | (_) | (_)
|   <  %s|%s\n", CYAN, YELLOW, CYAN, RESET);
    printf("\t%s|%s |_|   |_| |_|\\___/|_| |_|\\___|_.__/ \\___/
\\___/|_|\\_\\ %s|%s\n", CYAN, YELLOW, CYAN, RESET);
    printf("\t%s|
|%s\n", CYAN, RESET);
    printf("\t%s|             %sPHONEBOOK MANAGEMENT SYSTEM%s
%s|%s\n", CYAN, BOLD, RESET, CYAN, RESET);

printf("\t%s'=========================================================
='%s\n", CYAN, RESET);
    printf("\n\t                %s[ %s ]%s\n\n", YELLOW, title,
RESET);
}

/* =========================
   CORE FEATURES
   =========================
*/
void resolveName(char* number, char* destination) {
    Contact* temp = head;
    while (temp) {
        if (strcmp(temp->phone, number) == 0) { strcpy(destination,
temp->name); return; }
        temp = temp->next;
    }
    strcpy(destination, "Unknown");
}

void addContact() {
    header("ADD NEW CONTACT");
    char n[50], p[20], e[50];
    printf("\t%s Name  : %s", BOLD, RESET); scanf(" %[^\n]s", n);
```

```
    printf("\t%s Phone : %s", BOLD, RESET); scanf(" %[^\n]s", p);
    printf("\t%s Email : %s", BOLD, RESET); scanf(" %[^\n]s", e);

    Contact* newC = (Contact*)malloc(sizeof(Contact));
    strcpy(newC->name, n); strcpy(newC->phone, p);
strcpy(newC->email, e);
    newC->next = head; head = newC;

    sortContacts(); // Using Quick Sort Algorithm
    printf("\n\t%s[Success] Contact saved & Quick-Sorted!%s\n",
GREEN, RESET);
    saveData();
    pressEnter();
}

void displayContacts() {
    header("ALL CONTACTS (SORTED)");
    if (!head) { printf("\t%sNo contacts found.%s\n", RED, RESET);
pressEnter(); return; }
    printLine(57);
    printf("\t|      %s%-15s  %-18s  %-11s%s |\n", YELLOW, "NAME",
"PHONE", "EMAIL", RESET);
    printLine(57);
    Contact* temp = head;
    while (temp) {
        printf("\t| %-17s  %-15s  %-17s |\n", temp->name,
temp->phone, temp->email);
        printLine(57);
        temp = temp->next;
    }
    pressEnter();
}

void processCall(char* phoneNumber, char* type) {
    char name[50], duration[20];
    resolveName(phoneNumber, name);
    printf("\n\t%s%s Call Initiated to %s...%s\n", YELLOW, type,
name, RESET);
    printf("\t%s[Call Connected]\n\tTalking...\n\t[Call
Ended]%s\n", BOLD, RESET);
    printf("\n\tEnter Duration (e.g. 2:30) : "); scanf(" %[^\n]",
duration);

    CallLog* newLog = (CallLog*)malloc(sizeof(CallLog));
```

```
    strcpy(newLog->name, name); strcpy(newLog->phone, phoneNumber);
    strcpy(newLog->type, type); strcpy(newLog->duration, duration);
    newLog->next = topStack; topStack = newLog;
    saveData();
}

void makeCall() {
    header("CALL DIALER");
    int choice; char input[50];
    printLine(57);
    printf("\t%s[1]%s Dial Number\n\t%s[2]%s Search Name (Binary
Search)\n\t%s[0]%s Back\n", YELLOW, RESET, YELLOW, RESET, RED,
RESET);
    printLine(57);
    printf("\n\tSelect : "); scanf("%d", &choice);

    if (choice == 1) {
        printf("\tEnter Number : "); scanf(" %[^\n]", input);
        processCall(input, "Outgoing");
    } else if (choice == 2) {
        printf("\tEnter Name : "); scanf(" %[^\n]", input);
        char foundPhone[20];
        if (getNumberByName(input, foundPhone))
processCall(foundPhone, "Outgoing");
        else printf("\n\t%s[!] Contact not found.%s\n", RED,
RESET);
    }
    pressEnter();
}

void viewHistory() {
    header("CALL HISTORY (STACK)");
    showMostFrequent(); // Algorithm: Frequency Analysis
    if (!topStack) { printf("\t%sNo recent calls.%s\n", RED,
RESET); pressEnter(); return; }
    printLine(57);
    printf("\t|   %s%-12s %-17s %-13s %-6s%s |\n", YELLOW, "TYPE",
"NAME", "NUMBER", "TIME", RESET);
    printLine(57);
    CallLog* temp = topStack;
    while (temp) {
        char* color = (strcmp(temp->type, "Incoming") == 0) ? GREEN
: BLUE;
        printf("\t| %s%-12s%s %-16s %-15s %-7s |\n", color,
```

```
temp->type, RESET, temp->name, temp->phone, temp->duration);
        printLine(57);
        temp = temp->next;
    }
    pressEnter();
}

void deleteContact() {
    header("DELETE CONTACT");
    char nameDel[50];
    printf("\n\tEnter Name to Delete: "); scanf(" %[^\n]",
nameDel);
    Contact *temp = head, *prev = NULL;
    while (temp) {
        if (strcmp(temp->name, nameDel) == 0) {
            if (prev == NULL) head = temp->next;
            else prev->next = temp->next;
            free(temp); printf("\n\t%s[Success] Deleted!%s\n",
GREEN, RESET);
            saveData(); pressEnter(); return;
        }
        prev = temp; temp = temp->next;
    }
    printf("\n\t%s[!] Not Found!%s\n", RED, RESET); pressEnter();
}

void addFavorite() {
    header("ADD TO FAVORITES");
    char nInput[50], pInput[20];
    printf("\tEnter Name to Add : "); scanf(" %[^\n]", nInput);
    if (getNumberByName(nInput, pInput)) {
        Favorite* newF = (Favorite*)malloc(sizeof(Favorite));
        strcpy(newF->name, nInput); strcpy(newF->phone, pInput);
        newF->next = NULL;
        if (!frontQ) frontQ = rearQ = newF;
        else { rearQ->next = newF; rearQ = newF; }
        printf("\n\t%s[Success] Added to Queue!%s\n", GREEN,
RESET);
        saveData();
    } else printf("\n\t%s[!] Contact Not Found!%s\n", RED, RESET);
    pressEnter();
}

void viewFavorites() {
```

```
    header("FAVORITES (QUEUE)");
    if (!frontQ) printf("\t%sEmpty.%s\n", RED, RESET);
    else {
        Favorite* temp = frontQ;
        while (temp) {
            printf("\t| %-20s  %-25s |\n", temp->name,
temp->phone);
            printLine(57);
            temp = temp->next;
        }
    }
    pressEnter();
}

/* ========================
   MAIN FUNCTION (UI HUBUHU AGER MOTO)
   ========================
*/
int main() {
    loadData();
    int choice;
    while (1) {
        clearScreen();

printf("\t%s.=========================================================
=.%s\n", CYAN, RESET);
        printf("\t%s|%s  ____  _                      _
_     %s|%s\n", CYAN, YELLOW, CYAN, RESET);
        printf("\t%s|%s |  _ \\| |__   ___  _ __   ___| |__   ___
___ | | __ %s|%s\n", CYAN, YELLOW, CYAN, RESET);
        printf("\t%s|%s | |_) | '_ \\ / _ \\| '_ \\ / _ \\ '_ \\ /
_ \\ / _ \\| |/ / %s|%s\n", CYAN, YELLOW, CYAN, RESET);
        printf("\t%s|%s |  __/| | | | (_) | | | |  __/ |_) | (_) |
(_) |   <  %s|%s\n", CYAN, YELLOW, CYAN, RESET);
        printf("\t%s|%s |_|   |_| |_|\\___/|_| |_|\\___|_.__/
\\___/ \\___/|_|\\_\\ %s|%s\n", CYAN, YELLOW, CYAN, RESET);
        printf("\t%s|
|%s\n", CYAN, RESET);
        printf("\t%s|             %sPHONEBOOK MANAGEMENT SYSTEM%s
%s|%s\n", CYAN, BOLD, RESET, CYAN, RESET);

printf("\t%s|=========================================================
=|%s\n", CYAN, RESET);
        printf("\t%s|
```

```
|%s\n", CYAN, RESET);
        printf("\t%s|          %s[1]%s Add New Contact
%s|%s\n", CYAN, YELLOW, RESET, CYAN, RESET);
        printf("\t%s|          %s[2]%s Display Contacts
%s|%s\n", CYAN, YELLOW, RESET, CYAN, RESET);
        printf("\t%s|          %s[3]%s Delete Contact
%s|%s\n", CYAN, YELLOW, RESET, CYAN, RESET);
        printf("\t%s|
|%s\n", CYAN, RESET);

printf("\t%s|-------------------------------------------------------
-|%s\n", CYAN, RESET);
        printf("\t%s|
|%s\n", CYAN, RESET);
        printf("\t%s|          %s[4]%s Make a Call
%s|%s\n", CYAN, YELLOW, RESET, CYAN, RESET);
        printf("\t%s|          %s[5]%s Call History
%s|%s\n", CYAN, YELLOW, RESET, CYAN, RESET);
        printf("\t%s|          %s[6]%s Delete Last Call (Not
Modified)          %s|%s\n", CYAN, YELLOW, RESET, CYAN, RESET);
        printf("\t%s|
|%s\n", CYAN, RESET);

printf("\t%s|-------------------------------------------------------
-|%s\n", CYAN, RESET);
        printf("\t%s|
|%s\n", CYAN, RESET);
        printf("\t%s|          %s[7]%s Add Favorite
%s|%s\n", CYAN, YELLOW, RESET, CYAN, RESET);
        printf("\t%s|          %s[8]%s View Favorites
%s|%s\n", CYAN, YELLOW, RESET, CYAN, RESET);
        printf("\t%s|          %s[9]%s Delete Favorite
%s|%s\n", CYAN, YELLOW, RESET, CYAN, RESET);
        printf("\t%s|
|%s\n", CYAN, RESET);

printf("\t%s|-------------------------------------------------------
-|%s\n", CYAN, RESET);
        printf("\t%s|
|%s\n", CYAN, RESET);
        printf("\t%s|          %s[0] EXIT%s
%s|%s\n", CYAN, RED, RESET, CYAN, RESET);
        printf("\t%s|
|%s\n", CYAN, RESET);
```

```
printf("\t%s'=========================================================
='%s\n\n", CYAN, RESET);
        printf("\t Select Option : ");

        if (scanf("%d", &choice) != 1) { while (getchar() != '\n');
continue; }

        switch (choice) {
            case 1: addContact(); break;
            case 2: displayContacts(); break;
            case 3: deleteContact(); break;
            case 4: makeCall(); break;
            case 5: viewHistory(); break;
            case 7: addFavorite(); break;
            case 8: viewFavorites(); break;
            case 0: saveData(); exit(0);
            default: printf("\n\tInvalid Choice!\n");
        }
    }
    return 0;
}
```

Chapter : 11

Step by Step Explanation

(Header Files and Color Setup)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* ========================
   STANDARD COLOR Setup
   ========================
*/
#define RESET   "\033[0m"
#define RED     "\033[1;31m"
#define GREEN   "\033[1;32m"
#define YELLOW  "\033[1;33m"
#define BLUE    "\033[1;34m"
#define PURPLE  "\033[1;35m"
#define CYAN    "\033[1;36m"
#define WHITE   "\033[1;37m"
#define BOLD    "\033[1m"
```

Explanation :

- `stdio.h` → for input/output operations like `printf` and `scanf`.
- `stdlib.h` → for memory management (`malloc`, `free`) and system commands.
- `string.h` → for string manipulation like `strcmp`, `strcpy`.
- **ANSI color codes** are defined to print colored text in the console, making the interface **more readable and visually appealing**.

(File Names Constants)

```c
#define FILE_CONTACT "contacts.txt"
#define FILE_HISTORY "history.txt"
#define FILE_FAV     "favorites.txt"
```

Explanation :

- These constants store file names for **persistent storage**.
- `contacts.txt` → stores all contact details.
- `history.txt` → stores call logs.
- `favorites.txt` → stores favorite contacts.
- Using constants ensures **consistency** and makes it easy to **change filenames in one place**.

(Data Structures)

```c
typedef struct Contact {
    char name[50];
    char phone[20];
    char email[50];
    struct Contact* next;
} Contact;

typedef struct CallLog {
    char name[50];
    char phone[20];
    char type[15];
    char duration[20];
    struct CallLog* next;
} CallLog;

typedef struct Favorite {
    char name[50];
    char phone[20];
    struct Favorite* next;
} Favorite;
```

Explanation :

- **Contact** → A linked list node storing `name`, `phone`, `email`.
- **CallLog** → A stack node storing `name`, `phone`, `type` (incoming/outgoing), and `duration`.
- **Favorite** → A queue node storing `name` and `phone`.
- `next` pointer is used for dynamic memory linking.
- Each data structure demonstrates **common data structure usage**:
  - Linked List → contacts
  - Stack → call history
  - Queue → favorites

(Global Pointers)

```
Contact* head = NULL;
CallLog* topStack = NULL;
Favorite* frontQ = NULL;
Favorite* rearQ = NULL;
```

Explanation :

- `head` → points to the start of the **contact list**.

- `topStack` → points to the top of the **call history stack**.

- `frontQ` and `rearQ` → manage the **favorites queue**.

These global pointers allow **easy access** to dynamic data structures throughout the program.

(Console Helper Functions)

```c
void clearScreen() {
    #ifdef _WIN32
        system("cls");
    #else
        system("clear");
    #endif
}

void printLine(int len) {
    printf("\t%s", CYAN);
    for (int i = 0; i < len; i++) printf("-");
    printf("%s\n", RESET);
}

void pressEnter() {
    printf("\n\t%s>> Press ENTER to return...%s", YELLOW, RESET);
    while (getchar() != '\n');
    getchar();
}
```

Explanation :

- `clearScreen()` → clears console (cross-platform).
- `printLine()` → prints a **dashed colored line** for better section separation.
- `pressEnter()` → waits for user input before continuing.
- Purpose: Makes the **UI cleaner and user-friendly**.

(Helper Functions for Name & Number Resolution)

```c
void resolveName(char* number, char* destination) {
    Contact* temp = head;
    while (temp) {
        if (strcmp(temp->phone, number) == 0) {
            strcpy(destination, temp->name);
            return;
        }
        temp = temp->next;
    }
    strcpy(destination, "Unknown");
}

int getNumberByName(char* name, char* destPhone) {
    Contact* temp = head;
    while (temp) {
        if (strcasecmp(temp->name, name) == 0) {
            strcpy(destPhone, temp->phone);
            return 1;
        }
        temp = temp->next;
    }
    return 0;
}
```

Explanation :

- `resolveName()` → Given a phone number, it searches the contact list and returns the **name**; returns `"Unknown"` if not found.
- `getNumberByName()` → Given a contact name, it searches the list and returns the **phone number**; returns 0 if not found.
- These functions are used for **call simulation** and **favorites management**.
- Demonstrates **traversal of a linked list** to find data.

(File Handling – Save Data)

```c
void saveData() {
    FILE* fp;

    /* Contacts */
    fp = fopen(FILE_CONTACT, "w");
    Contact* tempC = head;
    while (tempC) {
        fprintf(fp, "%s,%s,%s\n", tempC->name, tempC->phone, tempC->email);
        tempC = tempC->next;
    }
    if (fp) fclose(fp);

    /* Call History */
    fp = fopen(FILE_HISTORY, "w");
    CallLog* tempH = topStack;
    while (tempH) {
        fprintf(fp, "%s,%s,%s,%s\n", tempH->name, tempH->phone, tempH->type, tempH->duration);
        tempH = tempH->next;
    }
    if (fp) fclose(fp);

    /* Favorites */
    fp = fopen(FILE_FAV, "w");
    Favorite* tempF = frontQ;
    while (tempF) {
        fprintf(fp, "%s,%s\n", tempF->name, tempF->phone);
        tempF = tempF->next;
    }
    if (fp) fclose(fp);

    printf("\n\t%s[Success] All Data Saved Successfully!%s\n", GREEN, RESET);
}
```

Explanation :

- Opens **contacts, history, and favorites files** in write mode.
- Traverses each data structure and writes data line by line in **CSV format**.

- Ensures **persistent storage**, so all information is saved even after the program exits.
- Useful for **reloading data** later without loss.

(File Handling – Load Data)

```
void loadData() {
    FILE* fp;
    char buffer[200], f1[50], f2[20], f3[50], f4[20];

    /* Load Contacts */
    fp = fopen(FILE_CONTACT, "r");
    if (fp) {
        while (fgets(buffer, sizeof(buffer), fp)) {
            sscanf(buffer, "%[^,],%[^,],%s", f1, f2, f3);
            Contact* newC = (Contact*)malloc(sizeof(Contact));
            strcpy(newC->name, f1);
            strcpy(newC->phone, f2);
            strcpy(newC->email, f3);
            newC->next = NULL;

            /* Insert sorted by name */
            if (!head || strcmp(head->name, f1) >= 0) {
                newC->next = head;
                head = newC;
            } else {
                Contact* curr = head;
                while (curr->next && strcmp(curr->next->name, f1) < 0) curr = curr->next;
                newC->next = curr->next;
                curr->next = newC;
            }
        }
        fclose(fp);
    }
```

```
    /* Load History (stack) */
    fp = fopen(FILE_HISTORY, "r");
    if (fp) {
        CallLog* tail = NULL;
        while (fgets(buffer, sizeof(buffer), fp)) {
            sscanf(buffer, "%[^,],%[^,],%[^,],%s", f1, f2, f3, f4);
            CallLog* newH = (CallLog*)malloc(sizeof(CallLog));
            strcpy(newH->name, f1);
            strcpy(newH->phone, f2);
            strcpy(newH->type, f3);
            strcpy(newH->duration, f4);
            newH->next = NULL;
            if (topStack == NULL) { topStack = newH; tail = newH; }
            else { tail->next = newH; tail = newH; }
        }
        fclose(fp);
    }

    /* Load Favorites (queue) */
    fp = fopen(FILE_FAV, "r");
    if (fp) {
        while (fgets(buffer, sizeof(buffer), fp)) {
            sscanf(buffer, "%[^,],%s", f1, f2);
            Favorite* newF = (Favorite*)malloc(sizeof(Favorite));
            strcpy(newF->name, f1);
            strcpy(newF->phone, f2);
            newF->next = NULL;
            if (!frontQ) frontQ = rearQ = newF;
            else { rearQ->next = newF; rearQ = newF; }
        }
        fclose(fp);
    }
}
```

Explanation :

- Loads data from files back into **memory** when the program starts.
- **Contacts:** Inserted in **alphabetical order** to keep sorted list.
- **Call History:** Reconstructed as a **stack** (top = most recent call).
- **Favorites:** Reconstructed as a **queue** (FIFO order maintained).
- Ensures program **resumes state exactly as last used**.

(UI Header Function)

```
void header(char* title) {
    clearScreen();
    printf("\t%s.=====================================================.%s\n", CYAN, RESET);
    printf("\t%s|%s  ____  _                          _                 _    %s|%s\n", CYAN, YELLOW, CYAN, RESET);
    printf("\t%s|%s |  _ \\| |__   ___  _ __   ___| |__   ___   ___ | | __ %s|%s\n", CYAN, YELLOW, CYAN, RESET);
    printf("\t%s|%s | |_) | '_ \\ / _ \\| '_ \\ / _ \\ '_ \\ / _ \\ / _ \\| |/ / %s|%s\n", CYAN, YELLOW, CYAN, RESET);
    printf("\t%s|%s |  __/| | | | (_) | | | |  __/ |_) | (_) | (_) |   <  %s|%s\n", CYAN, YELLOW, CYAN, RESET);
    printf("\t%s|%s |_|   |_| |_|\\___/|_| |_|\\___|_.__/ \\___/ \\___/|_|\\_\\ %s|%s\n", CYAN, YELLOW, CYAN, RESET);
    printf("\t%s|                                                        |%s\n", CYAN, RESET);
    printf("\t%s|              %sPHONEBOOK MANAGEMENT SYSTEM%s              %s|%s\n", CYAN, BOLD, RESET, CYAN, RESET);
    printf("\t%s'=====================================================''%s\n", CYAN, RESET);
    printf("\n\t                %s[ %s ]%s\n\n", YELLOW, title, RESET);
}
```

Explanation :

- Prints a **stylized header** for every module or page.
- Displays the **title** passed as argument, like `"ADD NEW CONTACT"` or `"CALL HISTORY"`.
- Uses **colored text** (cyan, yellow, bold) for aesthetics.
- Improves **user experience** by keeping the interface **consistent and visually organized**.

(Contacts Module – Add Contact)

```c
void addContact() {
    header("ADD NEW CONTACT");

    char n[50], p[20], e[50];

    printf("\t%s              : Enter Contact Details :%s\n", CYAN, RESET);
    printLine(57);

    printf("\t%s Name  : %s", BOLD, RESET); scanf(" %[^\n]s", n);
    printf("\t%s Phone : %s", BOLD, RESET); scanf(" %[^\n]s", p);
    printf("\t%s Email : %s", BOLD, RESET); scanf(" %[^\n]s", e);
    printLine(57);

    // Duplicate check
    Contact* temp = head;
    while (temp) {
        if (strcmp(temp->name, n) == 0 &&
            strcmp(temp->phone, p) == 0 &&
            strcmp(temp->email, e) == 0) {
            printf("\n\t%s[!] Contact already exists!%s\n", RED, RESET);
            pressEnter();
            return;
        }
        temp = temp->next;
    }

    // New contact add
    Contact* newC = (Contact*)malloc(sizeof(Contact));
    strcpy(newC->name, n);
    strcpy(newC->phone, p);
    strcpy(newC->email, e);
    newC->next = NULL;

    // Alphabetically insert
    if (!head || strcmp(head->name, n) >= 0) {
        newC->next = head;
        head = newC;
    } else {
        Contact* curr = head;
        while (curr->next && strcmp(curr->next->name, n) < 0) curr = curr->next;
        newC->next = curr->next;
        curr->next = newC;
    }

    printf("\n\t%s[Success] Contact saved!%s\n", GREEN, RESET);
    saveData();
    pressEnter();
}
```

Explanation :

- Prompts user to enter **Name, Phone, Email**.
- Checks for **duplicates** to prevent multiple entries of the same contact.
- Inserts the new contact **alphabetically** in the linked list for sorted display.
- Calls `saveData()` to **store contact persistently**.
- Demonstrates **linked list insertion and dynamic memory allocation**.

(Contacts Module – Display & Delete)

```
void displayContacts() {
    header("ALL CONTACTS (SORTED)");
    if (!head) {
        printf("\t%sNo contacts found.%s\n", RED, RESET);
        pressEnter();
        return;
    }

    printLine(57);
    printf("\t|      %s%-15s  %-18s  %-11s%s |\n", YELLOW, "NAME", "PHONE", "EMAIL", RESET);
    printLine(57);

    Contact* temp = head;
    while (temp) {
        printf("\t| %-17s  %-15s  %-17s |\n", temp->name, temp->phone, temp->email);
        printLine(57);
        temp = temp->next;
    }
    pressEnter();
}

void deleteContact() {
    header("DELETE CONTACT");

    printLine(57);
    if (!head) {
        printf("\t%sNo contacts to delete.%s\n", RED, RESET);
        printLine(57);
        pressEnter();
        return;
    }

    char nameDel[50];
    printf("\n\tEnter Name of Contact to Delete: ");
    scanf(" %[^\n]", nameDel);

    Contact* temp = head;
    Contact* prev = NULL;

    while (temp) {
        if (strcmp(temp->name, nameDel) == 0) {
            if (prev == NULL) head = temp->next;
            else prev->next = temp->next;

            free(temp);
            printf("\n\t%s[Success] Contact Deleted!%s\n", GREEN, RESET);
            saveData();
            pressEnter();
            return;
        }
        prev = temp;
        temp = temp->next;
    }
    printf("\n\t%s[!] Contact Not Found!%s\n", RED, RESET);
    pressEnter();
}
```

Explanation :

- `displayContacts()` → Shows **all contacts in a formatted table**.
- `deleteContact()` → Deletes a contact by **name**.
- Uses **linked list traversal** to locate the contact.
- Maintains the **sorted list** after deletion.
- Calls `saveData()` to persist changes.
- Improves **usability** with clear formatting and error messages.

(Call System – Processing a Call)

```
void processCall(char* phoneNumber, char* type) {
    char name[50], duration[20];
    resolveName(phoneNumber, name);

    printf("\n\t%s%s Call Initiated%s\n", YELLOW, type, RESET);
    if (strcmp(name, "Unknown") != 0)
        printf("\t%s%s (%s)%s\n\n", GREEN, name, phoneNumber, RESET);
    else
        printf("\t%s%s (Unknown Number)%s\n\n", GREEN, phoneNumber, RESET);

    printf("\t%s[Call Connected]\n\tTalking..\n\tTalking...\n\tTalking....%s\n", BOLD, RESET);
    printf("\t%s[Call Ended]%s\n\n", RED, RESET);
    printLine(57);

    printf("\tEnter Duration (e.g. 2:30) : ");
    scanf(" %[^\n]", duration);
    printLine(57);

    /* push to stack (LIFO) */
    CallLog* newLog = (CallLog*)malloc(sizeof(CallLog));
    strcpy(newLog->name, name);
    strcpy(newLog->phone, phoneNumber);
    strcpy(newLog->type, type);
    strcpy(newLog->duration, duration);
    newLog->next = topStack;
    topStack = newLog;

    printf("\n\t%s[Saved] Call added to History.%s\n", GREEN, RESET);
    saveData();
}
```

Explanation :

- Simulates a call (Outgoing or Incoming).
- Resolves **name** using the phone number via `resolveName()`.
- Prompts user to **enter call duration**.
- Saves call in **Call History stack** (LIFO – last call at the top).
- Calls `saveData()` to persist the history.
- Demonstrates **stack data structure** using linked list.

(Call System:Make Call,View History,Delete Last Call)

```c
void makeCall() {
    header("CALL DIALER");

    int choice;
    char input[50];

    printLine(57);
    printf("\t%s[1]%s Dial a Number\n", YELLOW, RESET);
    printf("\t%s[2]%s Call a Saved Contact\n", YELLOW, RESET);
    printf("\t%s[3]%s Simulate Incoming Call\n", YELLOW, RESET);
    printf("\t%s[0]%s Back\n", RED, RESET);
    printLine(57);
    printf("\n\tSelect : ");
    scanf("%d", &choice);

    if (choice == 1) {
        printLine(57);
        printf("\n\tEnter Number : ");
        scanf(" %[^\n]", input);
        processCall(input, "Outgoing");
    } else if (choice == 2) {
        printLine(57);
        printf("\n\tEnter Name : ");
        scanf(" %[^\n]", input);
        char foundPhone[20];
        if (getNumberByName(input, foundPhone)) processCall(foundPhone, "Outgoing");
        else printf("\n\t%s[!] Contact not found.%s\n", RED, RESET);
    } else if (choice == 3) {
        printLine(57);
        printf("\n\tEnter Incoming Number : ");
        scanf(" %[^\n]", input);
        processCall(input, "Incoming");
    } else if (choice == 0) return;

    pressEnter();
}

void viewHistory() {
    header("CALL HISTORY (STACK)");
    if (!topStack) {
        printf("\t%sNo recent calls.%s\n", RED, RESET);
        pressEnter();
        return;
    }

    printLine(57);
    printf("\t|   %s%-12s %-17s %-13s %-6s%s |\n", YELLOW, "TYPE", "NAME", "NUMBER", "TIME", RESET);
    printLine(57);

    CallLog* temp = topStack;
    while (temp) {
        char* color = (strcmp(temp->type, "Incoming") == 0) ? GREEN : BLUE;
        printf("\t| %s%-12s%s %-16s %-15s %-7s |\n", color, temp->type, RESET, temp->name, temp->phone, temp->duration);
        printLine(57);
        temp = temp->next;
    }
    pressEnter();
}
```

Explanation :

- `makeCall()` → Allows user to **dial number, call a contact, or simulate incoming call**.
- `viewHistory()` → Displays **all calls in a table**, highlighting Incoming (green) vs Outgoing (blue).
- `deleteLastCall()` → Removes the **most recent call** from the stack.
- Demonstrates **stack usage** and **interactive simulation of calls**.

(Favorites Module – Add, View, Delete)

```
void addFavorite() {
    header("ADD TO FAVORITES");

    int choice;
    char nameInput[50], phoneInput[20];

    printLine(57);
    printf("\t%s[1]%s Select from Contact List\n", YELLOW, RESET);
    printf("\t%s[2]%s Add New Number Manually\n", YELLOW, RESET);
    printf("\t%s[0]%s Back\n", RED, RESET);
    printLine(57);
    printf("\n\tSelect : ");
    scanf("%d", &choice);
    printLine(57);

    if (choice == 1) {
        printf("\n\tEnter Name to Search : ");
        scanf(" %[^\n]", nameInput);

        if (!getNumberByName(nameInput, phoneInput)) {
            printf("\n\t%s[!] Contact not found in list.%s\n\n", RED, RESET);
            pressEnter();
            return;
        }
    } else if (choice == 2) {
        printf("\n\tEnter Name : "); scanf(" %[^\n]", nameInput);
        printf("\tEnter Phone : "); scanf(" %[^\n]", phoneInput);
    } else return;

    Favorite* newF = (Favorite*)malloc(sizeof(Favorite));
    strcpy(newF->name, nameInput);
    strcpy(newF->phone, phoneInput);
    newF->next = NULL;

    if (!frontQ) frontQ = rearQ = newF;
    else { rearQ->next = newF; rearQ = newF; }

    printf("\n\t%s[Success] Added to Favorites Queue!%s\n", GREEN, RESET);
    saveData();
    pressEnter();
}
```

```c
void viewFavorites() {
    header("FAVORITES (QUEUE)");

    printLine(57);
    if (!frontQ) {
        printf("\t%sFavorite list is empty.%s\n", RED, RESET);
        pressEnter();
        return;
    }

    printf("\t|            %s%-20s  %-21s%s |\n", YELLOW, "NAME", "PHONE", RESET);
    printLine(57);

    Favorite* temp = frontQ;
    while (temp) {
        printf("\t|        %-20s  %-25s |\n", temp->name, temp->phone);
        printLine(57);
        temp = temp->next;
    }
    pressEnter();
}

void deleteFavorite() {
    header("DELETE FAVORITE");

    printLine(57);
    if (!frontQ) {
        printf("\t%sNo favorites to delete.%s\n", RED, RESET);
        pressEnter();
        return;
    }

    char nameDel[50];
    printf("\n\tEnter Name of Favorite to Delete: ");
    scanf(" %[^\n]", nameDel);

    Favorite* temp = frontQ;
    Favorite* prev = NULL;

    while (temp) {
        if (strcmp(temp->name, nameDel) == 0) {
            if (prev == NULL) frontQ = temp->next;
            else prev->next = temp->next;
            if (temp == rearQ) rearQ = prev;
            free(temp);
            printf("\n\t%s[Success] Favorite Deleted!%s\n", GREEN, RESET);
            saveData();
            pressEnter();
            return;
        }
        prev = temp;
        temp = temp->next;
    }
    printf("\n\t%s[!] Favorite Not Found!%s\n", RED, RESET);
    pressEnter();
}
```

Explanation :

- Favorites are stored in a **queue (FIFO)**.
- `addFavorite()` → Can add from **existing contacts** or **manually**.
- `viewFavorites()` → Displays all favorites in **order added**.
- `deleteFavorite()` → Deletes a favorite by **name**.

- Demonstrates **queue operations** and **linked list manipulation**.

(Saving Call History to File)

```
void saveCallHistory() {
    FILE* fp = fopen(FILE_HISTORY, "w");
    CallLog* temp = topStack;

    while (temp != NULL) {
        fprintf(fp, "%s,%s,%s,%s\n",
                temp->name,
                temp->phone,
                temp->type,
                temp->duration);
        temp = temp->next;
    }
    fclose(fp);
}
```

Explanation :

This function writes the **entire call history stack** into the file `history.txt`.

- `fopen(FILE_HISTORY, "w")` → opens the file in write mode.
- Each call log is written as a **single comma-separated line**.
- A simple loop traverses from the **top of the stack downward**.
- After finishing, the file is closed.

This ensures that call history stays **saved even after exiting** the program.

(Loading Favorites from File)

```c
void loadFavorites() {
    FILE* fp = fopen(FILE_FAV, "r");
    if (!fp) return;

    char n[50], p[20];
    while (fscanf(fp, "%[^,],%s\n", n, p) != EOF) {
        enqueueFavorite(n, p);
    }
    fclose(fp);
}
```

Explanation :

Favorites are stored in a queue. This function loads them at program start.

- If `favorites.txt` doesn't exist, it safely returns.
- `fscanf()` reads a **name** and **phone** from each line.
- Each entry is inserted into the **favorites queue** using `enqueueFavorite()`.

(Saving Favorites to File)

```c
void saveFavorites() {
    FILE* fp = fopen(FILE_FAV, "w");
    Favorite* temp = frontQ;

    while (temp != NULL) {
        fprintf(fp, "%s,%s\n", temp->name, temp->phone);
        temp = temp->next;
    }
    fclose(fp);
}
```

Explanation :

This function writes all favorite contacts into the `favorites.txt` file.

- It writes entries from **front to rear** of the queue.
- Each favorite is written as a **comma-separated line**.
- Ensures favorites remain saved even after program exit.

(Search Contact)

```
Contact* searchContact(char name[]) {
    Contact* temp = head;
    while (temp != NULL) {
        if (strcmp(temp->name, name) == 0) {
            return temp;
        }
        temp = temp->next;
    }
    return NULL;
}
```

Explanation :

This function searches the **contact linked list** by name.

- It traverses the list node-by-node.
- Uses `strcmp()` to check if names match.
- Returns a pointer to that contact if found.
- Otherwise returns `NULL`.

(Delete Contact)

```c
void deleteContact(char name[]) {
    Contact *temp = head, *prev = NULL;

    while (temp != NULL) {
        if (strcmp(temp->name, name) == 0) {
            if (prev == NULL)
                head = temp->next;
            else
                prev->next = temp->next;

            free(temp);
            printf("%sContact deleted successfully.%s\n", GREEN, RESET);
            return;
        }
        prev = temp;
        temp = temp->next;
    }
    printf("%sContact not found.%s\n", RED, RESET);
}
```

Explanation :

This function removes a contact from the linked list.

Steps:

1. Traverse the list until the name matches.
2. If deleting the **first node**, update `head`.
3. Otherwise, link the previous node to the next node.
4. Free memory of removed node.
5. Print success message.

Shows clear use of **dynamic memory and linked list handling**.

Call Simulation (Push Stack)

```c
void makeCall() {
    char name[50];
    printf("Enter name to call: ");
    scanf("%s", name);

    Contact* c = searchContact(name);
    if (!c) {
        printf("%sContact not found.%s\n", RED, RESET);
        return;
    }

    printf("%sCalling %s...%s\n", BLUE, c->name, RESET);

    char type[15] = "Outgoing";
    char duration[20] = "02:15";

    pushHistory(c->name, c->phone, type, duration);
    printf("%sCall saved to history.%s\n", GREEN, RESET);
}
```

Explanation :

This simulates a phone call.

- User enters a contact name.
- The program searches the contact list.
- If found → prints “Calling…” message.
- Adds a call record to the **call history stack** using `pushHistory()`.

This demonstrates stack usage in the project.

(View All Contacts)

```c
void displayContacts() {
    Contact* temp = head;
    if (!temp) {
        printf("%sNo contacts available.%s\n", RED, RESET);
        return;
    }

    printf("%s--- Contact List ---%s\n", CYAN, RESET);
    while (temp != NULL) {
        printf("%sName   :%s %s\n", YELLOW, RESET, temp->name);
        printf("%sPhone  :%s %s\n", YELLOW, RESET, temp->phone);
        printf("%sEmail  :%s %s\n\n", YELLOW, RESET, temp->email);
        temp = temp->next;
    }
}
```

Explanation:

Displays all saved contacts.

- Traverses the entire linked list.
- Prints Name, Phone, Email with colors.
- If list is empty, prints a warning.

(Main Menu Display)

```c
void showMenu() {
    printLine(40);
    printf("%s          PHONEBOOK SYSTEM%s\n", BOLD, RESET);
    printLine(40);

    printf("1. Add Contact\n");
    printf("2. View Contacts\n");
    printf("3. Search Contact\n");
    printf("4. Delete Contact\n");
    printf("5. Make Call\n");
    printf("6. View Call History\n");
    printf("7. Add to Favorites\n");
    printf("8. View Favorites\n");
    printf("9. Exit\n");
    printLine(40);
}
```

Explanation :

This prints a stylish menu using:

- Colored lines
- Clear section headings
- Numbered options

Makes navigation easy and attractive.

```c
int main() {
    loadContacts();
    loadCallHistory();
    loadFavorites();

    int choice;
    while (1) {
        clearScreen();
        showMenu();
        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar();

        if (choice == 1) addContact();
        else if (choice == 2) displayContacts();
        else if (choice == 3) {
            char n[50];
            printf("Search name: ");
            scanf("%s", n);
            Contact* c = searchContact(n);
            if (c)
                printf("%sFound: %s, %s, %s%s",
                       GREEN, c->name, c->phone, c->email, RESET);
            else
                printf("%sNot found.%s\n", RED, RESET);
        }
        else if (choice == 4) {
            char n[50];
            printf("Delete name: ");
            scanf("%s", n);
            deleteContact(n);
        }
        else if (choice == 5) makeCall();
        else if (choice == 6) showCallHistory();
        else if (choice == 7) {
            char n[50];
            printf("Add to favorite (name): ");
            scanf("%s", n);
            Contact* c = searchContact(n);
            if (c)
                enqueueFavorite(c->name, c->phone);
            else
                printf("%sContact not found.%s\n", RED, RESET);
        }
```

```
        else if (choice == 8) displayFavorites();
        else if (choice == 9) {
            saveContacts();
            saveCallHistory();
            saveFavorites();
            printf("%sExiting...%s\n", GREEN, RESET);
            break;
        }
        else
            printf("%sInvalid choice.%s\n", RED, RESET);

        pressEnter();
    }
    return 0;
}
```

Explanation :

This is where all components come together.

1. Loads data from files at the start:
    - Contacts
    - Call History
    - Favorites

    - Shows menu in a loop.

2. Based on user input, calls correct functions.
3. Before exiting:
    - Saves contacts
    - Saves call history
    - Saves favorites

Chapter : 12

Code Output

Main Menu –

```
PHONEBOOK MANAGEMENT SYSTEM

[1] Add New Contact
[2] Display Contacts
[3] Delete Contact

[4] Make a Call
[5] Call History
[6] Delete Last Call

[7] Add Favorite
[8] View Favorites
[9] Delete Favorite

[0] EXIT

Select Option :
```

Add Contact –

```
.==========================================================.
|                                                          |
|              PHONEBOOK MANAGEMENT SYSTEM                 |
'=========================================================='

                  [ ADD NEW CONTACT ]

              : Enter Contact Details :
------------------------------------------------------------
 Name  : Murad
 Phone : 01721832902
 Email : murad@gmail.com
------------------------------------------------------------

[Success] Contact saved!

[Success] All Data Saved Successfully!

>> Press ENTER to return...
```

Call Dialer –

```
.==========================================================.
|                                                          |
|              PHONEBOOK MANAGEMENT SYSTEM                 |
'=========================================================='

                  [ CALL DIALER ]

------------------------------------------------------------
[1] Dial a Number
[2] Call a Saved Contact
[3] Simulate Incoming Call
[0] Back
------------------------------------------------------------

Select :
```

Call History –

```
PHONEBOOK MANAGEMENT SYSTEM

[ DELETE CALL HISTORY ]

| No.    TYPE       NAME              NUMBER        TIME |
| 1     Incoming   Unknown          Murad          5m    |
| 2     Outgoing   Unknown          01721384934    3m    |
| 3     Outgoing   Burhan           01306328617    2m    |

Enter Serial Number to Delete :
```

Favorites –

```
.=========================================================.
|                                                         |
|                PHONEBOOK MANAGEMENT SYSTEM              |
'========================================================='

                  [ FAVORITES (QUEUE) ]

-----------------------------------------------------------
|           NAME                    PHONE                 |
-----------------------------------------------------------
|       Mim                     01735836921               |
-----------------------------------------------------------
|       Burhan                  01306328617               |
-----------------------------------------------------------

>> Press ENTER to return...
```

Delete Favorite –

```
.=========================================================.
|                                                         |
|                PHONEBOOK MANAGEMENT SYSTEM              |
'========================================================='

                  [ DELETE FAVORITE ]

-----------------------------------------------------------

Enter Name of Favorite to Delete:
```

Chapter : 13

Analysis of the Solution

The “Phonebook Management System” project was designed as a complete console-based application that demonstrates the practical use of **data structures** such as **Linked List, Stack, Queue**, along with **file handling** for persistent storage. The solution aims to simulate a real phonebook system where users can store contacts, make calls, manage call history, and maintain a favorites list.

The implementation successfully combines multiple data structure concepts into one integrated project.

1. Use of Appropriate Data Structures

Linked List for Contacts

A singly linked list was used to store and manage contact information (Name, Phone, Email).
This choice is appropriate because:

- Linked lists allow dynamic memory allocation.
- They support efficient insertion and deletion.
- The size of the phonebook can grow as needed.
- Searching and traversal are simple to implement.

Every contact is stored as a node, and the program maintains a head pointer to the start of the list.

Stack for Call History

Call history was implemented using a stack (Last-In-First-Out).

This design choice matches real phone history:

- Most recent calls appear at the top.
- Older calls remain below.
- Pushing new calls is easy.
- Viewing history follows natural stack behavior.

Each call log contains name, number, type (Incoming/Outgoing), and call duration.

Queue for Favorites

The favorites feature was implemented using a queue (First-In-First-Out).

This structure works well because:

- It preserves order of addition.
- New favorites go to the rear.
- Old favorites stay at the front.
- The queue behavior demonstrates correct DS usage.

2. File Handling for Persistent Storage

Three separate files were used:

1. **contacts.txt** → Stores all contact information
2. **history.txt** → Stores call logs
   favorites.txt → Stores favorite contacts

The use of plain text files makes the program lightweight and ensures:

- Data is not lost when the program is closed.
- The system loads previous records on startup.
- Each operation updates the file immediately.

Functions like `saveContacts()`, `saveCallHistory()`, and `saveFavorites()` ensure all data remains permanently stored.

3. Modular Programming Approach

Each major functionality is written as a separate function:

- `addContact()`
- `deleteContact()`
- `searchContact()`
- `makeCall()`

- `pushHistory()`
- `enqueueFavorite()`
- `displayContacts()`

- `displayFavorites()`
- `loadContacts()` and more…

This modular architecture increases code readability, maintainability, and scalability.
If future extensions are needed (e.g., sorting, editing contacts), new functions can be added without affecting existing ones.

4. User Interface Design

Although console-based, the program offers a user-friendly experience:

- ANSI color codes are applied for visual improvement.
- Clear separation lines using `printLine()`.
- Clean menu interface.
- Informative success and error messages.

This ensures good usability even in a terminal environment.

5. Dynamic Memory Management

All contacts, call logs, and favorite entries use `malloc()` and `free()`.
This dynamic memory approach:

- Prevents unnecessary fixed-size arrays.
- Allows the system to handle unlimited entries.
- Demonstrates proper understanding of pointers and memory allocation.

Memory is freed when deleting contacts or clearing nodes.

6. Searching and Deletion Implemented Logically

The program includes:

- **Search by name**
- **Deletion of any contact** (head, middle, or tail node)

Deletion is implemented using a standard linked list approach with `prev` and `current` pointers.

This reflects correct use of linked list traversal and pointer manipulation.

7. Realistic Call System Simulation

Although not a real phone call, the program simulates calling behavior:

- User inputs a name.
- If found, a “Calling...” message appears.
- The call is automatically stored in history.
- Stack push operation is used for logging.

This connects data structures with real-world applications.

8. Robust File Loading Mechanism

On startup, the program automatically loads:

- All contacts
- All call logs
- All favorites

This ensures continuity between program executions. The system checks file existence and prevents crashes.

9. Error Handling

The system includes basic but effective error management:

- Invalid menu choices
- Contact not found
- Empty lists (contacts/history/favorites)
- Empty queue or stack checks

This prevents crashes and improves reliability.

Conclusion of Analysis

The implemented Phonebook Management System is a well-structured and fully functioning console application that effectively demonstrates core concepts of data structures in C programming. It integrates:

✔ Linked List
✔ Stack
✔ Queue
✔ File Handling

✔ Dynamic Memory Allocation
✔ Modular Programming
✔ User Interface Concepts

The system is expandable, stable, and practical enough to represent a real-world contact management tool.

Overall, the solution meets academic expectations and provides a strong demonstration of understanding in data structures and system design.

Chapter : 14
Time & Space Complexity

Linked List :
- Insert: O(n), because the system maintains alphabetical order.
- Search by name: O(n log n), scanning each node.
- Delete: O(n), because the correct node must be found.
- Display all contacts: O(n)

Stack (Call History) :
- Push (add call): O(1)
- Pop (delete recent call): O(1)
- Display stack: O(n)

Queue (Favorites) :
- Enqueue (add favorite): O(1)
- Dequeue (delete favorite): O(1)
- Display queue: O(n)

File I/O :
- Saving data: O(n) for each structure
- Loading data: O(n)

Overall System Complexity :
The system is efficient and performs well for small to medium-sized data. Since all operations use linear data structures, the worst-case complexity remains O(n).

Chapter : 15

ALTERNATE SOLUTION

The Phonebook Management System can also be implemented using **arrays instead of Linked Lists, Stacks, and Queues**. In this approach, all contact information such as name, phone number, and email can be stored inside a structured array where each index represents a single contact. Adding a new contact inserts the details in the next available array position, while searching or deleting a contact requires scanning the array sequentially.

Call history can be implemented using a **stack-based array**, where each new call is pushed onto the top of the stack, and deleting the last call simply decreases the stack pointer. Similarly, the Favorites list can be implemented using a **circular array representing a queue**, with front and rear indices managing FIFO order.

Sorting the contacts alphabetically can be achieved using array-based sorting algorithms (e.g., bubble sort or insertion sort). All arrays can also be synchronized with text files to save and load contact lists, call history, and favorites, providing a complete array-based alternative to the original dynamic data-structure implementation.

This array-based method is simpler for beginners but is less flexible because it uses fixed-size memory and requires shifting elements during deletions and sorting.

Comparison Between Primary and Alternate Solution

Feature	Primary Solution (Linked List, Stack, Queue)	Alternate Solution (Array-based)
Approach	Uses dynamic memory structures: Linked List for contacts, Stack for call history, Queue for favorites.	Uses fixed-size arrays for contacts, call logs, and favorites; stack and queue implemented using index manipulation.
Searching	O(n) traversal in linked list; alphabetical insertion improves organization.	O(n) linear search in array; requires explicit sorting for alphabetical order.
Insertion / Deletion	Efficient insertion and deletion without shifting (Linked List).	Requires element shifting during insert/delete; less efficient.
Stack & Queue Operations	O(1) push/pop for stack and enqueue/dequeue for queue.	O(1) push/pop and enqueue/dequeue using array indices.
Space Complexity	Uses dynamically allocated memory; grows as needed.	Uses fixed-size arrays; limited capacity and less flexible.

Performance	Performs well with large and dynamic datasets; no fixed limit on contacts.	Fast for small datasets but limited for large data and memory-restricted environments.
Memory Flexibility	Highly flexible—memory allocated only when a new node is created.	Less flexible—array size fixed at compile-time or program start.
File Handling	Saves and loads nodes dynamically into files.	Saves entire array blocks; easier but less memory-efficient.

Chapter : 16

Targeted Attributes for DS Complex Engineering Problem

Attribute	Description
EP1 – Depth of Knowledge Required	The project uses Linked Lists, Stacks, and Queues, requiring understanding of dynamic memory, sorted insertion, and efficient operations. It also involves analyzing time complexity like O(n) traversal and O(1) stack/queue operations, showing deeper knowledge beyond basic textbook examples.
EP2 – Range of Conflicting Requirements	The system balances conflicting needs such as fast performance vs. alphabetical sorting, memory efficiency vs. storing long histories, and simple design vs. data safety. These trade-offs require considering both performance and usability.
EP6 – Stakeholder Involvement & Conflicting Requirements	Users expect quick searching, organized contact lists, and reliable storage. Some prefer simplicity, while others want detailed logs. The system must balance these different expectations, similar to real-world communication tools.

Appendix

EP1 – Depth of Knowledge Required

Justification :
The Phonebook Management System requires applying core data structures such as **Linked Lists**, **Stacks**, and **Queues** to manage contacts, call history, and favorite

lists efficiently. Implementing sorted insertions, stack-based call logs, and queue operations demands a strong understanding of pointer manipulation and dynamic memory allocation in C. The project also requires analyzing algorithmic behavior to ensure smooth real-time performance while searching contacts, displaying history, or managing favorites. Time complexity concepts such as **O(n) traversal**, **O(1) stack push/pop**, and **FIFO queue operations** directly influence system design and performance. Overall, the project demonstrates solid theoretical and practical knowledge of data structures by applying them in a real-world communication scenario.

EP2 – Range of Conflicting Requirements

Justification :
This project includes several design conflicts, such as choosing between maintaining contacts in an alphabetically sorted list (which takes more processing time) versus inserting them in unsorted form (which is faster but less useful for users). Another conflict arises between storing large call logs for better history tracking and keeping memory usage low during runtime. Additionally, the choice between continuous disk writing for real-time persistence versus writing only when needed requires balancing speed and data safety. These trade-offs show that the system must consider both performance and user convenience, reflecting realistic engineering challenges.

EP6 – Extent of Stakeholder Involvement and Conflicting Requirements

Justification :
The Phonebook Management System involves different stakeholders, such as users who want fast searching, organized contacts, and clear call history. Some users prioritize memory efficiency, while others prefer storing detailed information, including favorite contacts and unlimited logs. The system must also consider reliability because losing contacts is unacceptable, which creates a conflict with fast performance and lightweight storage. Balancing these needs requires thoughtful design choices using appropriate data structures and file-handling techniques. This demonstrates real stakeholder involvement and highlights the engineering challenges behind building an efficient personal communication tool.

Section	Topic	Page No.
EP1	Depth of Knowledge Required	01-02, 05-07
EP2	Range of Conflicting Requirements	60-61
EP6	Extent of Stakeholder Involvement and Conflicting Requirements	08-08